

Decoupled Static Indexing and Parameterized Search for Approximate MIPS over Learned Sparse Representations

Roger Canal Estrada

Universitat Politècnica de Catalunya (UPC)
Facultat d'Informàtica de Barcelona (FIB)
`roger.canal@estudiantat.upc.edu`

Abstract. Learned Sparse Representations (LSR), such as SPLADE, preserve lexical interpretability but induce an extreme Maximum Inner Product Search (MIPS) latency bottleneck in high-dimensional sparse spaces. This paper presents a decoupled architecture with two explicit phases: a high-capacity static index partitioned by document clustering, and a parameterized online search engine based on Coordinate-at-a-Time (CaaT) traversal. The static phase preserves the candidate pool with large fixed index capacity, while the dynamic phase controls latency–recall trade-offs through runtime constraints. On the Natural Questions benchmark (2.68M documents, 30,522 dimensions), the system reaches the target $\text{Recall@30} \geq 0.90$ at approximately 40 ms average latency and exposes direct search-time tuning levers that further reduce latency without index rebuilding.

Keywords: Learned Sparse Retrieval · Approximate Nearest Neighbor Search · Dynamic Pruning · Inverted Indexing

1 Introduction

LSR models map semantics into vocabulary space, so retrieval remains sparse and interpretable [2]. However, LSR query weights do not follow the same heavy-head Zipfian pattern that classical lexical pipelines exploit. Instead, many coordinates carry moderate but non-negligible mass (often described as *wacky weights*) [4]. This geometry weakens the pruning selectivity of traditional WAND-like and Document-at-a-Time (DaaT) traversal because partial-score upper bounds become less decisive at early stages [1].

This work addresses this limitation with block-level reasoning. The key idea is to decompose each posting list into geometrically coherent blocks and score blocks through tight upper bounds before touching documents. This strategy shifts computation from expensive sparse dot products to inexpensive block-level comparisons.

The paper contributes the following elements. First, it defines a decoupled architecture where a high-capacity static index preserves coverage and a parameterized dynamic search loop controls runtime cost. Second, it enforces exact-maximum summary bounds at index construction, which makes block upper

bounds mathematically safe for aggressive skipping. Third, it demonstrates that this separation yields a stable operating point above Recall@30 = 0.90 while preserving independent search-time control of latency.

2 System Architecture

The system is split into two phases: a *Static Index Phase* and a *Parameterized Search Phase*. The first phase maximizes representational coverage. The second phase imposes strict runtime budgets over the fixed structure.

2.1 Static Index Phase

Document Clustering Step Goal and math. The method partitions the document set $\mathcal{D} = \{d_i\}_{i=1}^N$ into K clusters by a sparse K-means objective:

$$\max_{\{\mathcal{C}_k\}, \{\mu_k\}} \sum_{k=1}^K \sum_{d_i \in \mathcal{C}_k} \langle d_i, \mu_k \rangle, \quad (1)$$

with assignment step

$$a(i) = \arg \max_{k \in \{1, \dots, K\}} \langle d_i, \mu_k \rangle, \quad (2)$$

and centroid update

$$\mu_k \leftarrow \frac{\sum_{d_i \in \mathcal{C}_k} d_i}{\left\| \sum_{d_i \in \mathcal{C}_k} d_i \right\|_2}. \quad (3)$$

Both operations are computed in sparse form.

Rationale. Clustering creates local geometric cohesion. Without this partition, each concept posting list mixes unrelated documents, and block-level upper bounds become loose. Loose bounds collapse skipping power. Cohesive clusters make upper bounds sharp enough to reject large low-potential regions.

Partitioned Inverted Index Construction Goal and math. The index maps each concept t to a list of **InvertedBlocks**. Each block stores one cluster id and truncated doc-weight pairs. For each cluster c , the method also builds a dense summary vector $S_c \in \mathbb{R}^{|V|}$ with

$$S_c[t] = \max_{d \in \mathcal{C}_c} d[t]. \quad (4)$$

This is the absolute maximum weight of concept t inside cluster c .

For each concept t , the index keeps at most nb cluster blocks and at most nd documents per retained block. These capacity parameters remain intentionally high in the static phase to preserve candidate coverage and maintain a high theoretical recall ceiling.

Algorithm 1 Static Index Construction with Exact-Maximum Summaries

Require: Sparse documents, cluster assignments, nb , nd
Ensure: Partitioned index I and summary vectors S

- 1: **for** each cluster c **do**
- 2: initialize dense summary $S_c \leftarrow 0$
- 3: **for** each document $d \in \mathcal{C}_c$ **do**
- 4: **for** each non-zero (t, w) in d **do**
- 5: $S_c[t] \leftarrow \max(S_c[t], w)$
- 6: **end for**
- 7: **end for**
- 8: **end for**
- 9: **for** each concept t **do**
- 10: aggregate candidate docs by cluster and concept weight
- 11: keep top- nb clusters by accumulated weight on t
- 12: **for** each retained cluster c **do**
- 13: keep top- nd docs by weight on t
- 14: append `InvertedBlock`($c, doc_ids, weights$) to $I[t]$
- 15: **end for**
- 16: **end for**
- 17: **return** I, S

Rationale. Exact maxima make the cluster upper bound

$$UB_c(q) = \sum_{t \in \text{supp}(q)} q[t] \cdot S_c[t] \quad (5)$$

mathematically valid (never under-estimated). This property prevents miss-skipping. If summaries under-estimate, the system can discard clusters that contain true top- k documents. The exact-maximum rule removes that failure mode.

2.2 Parameterized Search Phase

Coordinate-at-a-Time (CaaT) Traversal Goal. The search engine processes incoming sparse queries over the static index with CaaT traversal. It parses non-zero query coordinates, sorts them by descending weight, and iterates coordinates first.

Rationale. CaaT is preferred over DaaT because it touches the highest-signal query terms early. This rapidly increases the min-heap threshold. A high threshold amplifies subsequent skipping decisions. In contrast, DaaT mixes low- and high-signal evidence across many documents before the threshold stabilizes. Algorithm 2 formalizes the CaaT loop, while classical DaaT-style traversal underlies the baseline score-at-a-time strategy discussed in [1].

Aggressive Dynamic Pruning Heuristics Goal and math. The pruning stage rejects blocks by the inequality

$$UB_c(q) < H.\min() \cdot \text{heap_factor}. \quad (6)$$

Algorithm 2 Parameterized CaaT Search

Require: Query q , index I , summaries S , top- k , max_query_terms ,
 max_search_blocks , $heap_factor$, md

Ensure: Top- k result heap H

- 1: $Q \leftarrow$ non-zero query coordinates sorted by weight descending
- 2: **if** $max_query_terms > 0$ **then**
- 3: truncate Q to first max_query_terms terms
- 4: **end if**
- 5: compute $UB_c(q)$ for all clusters c
- 6: initialize min-heap H , visited set, $docs_examined \leftarrow 0$
- 7: **for** each term $t \in Q$ **do**
- 8: $blocks_seen \leftarrow 0$
- 9: **for** each block $b \in I[t]$ **do**
- 10: **if** $max_search_blocks > 0$ and $blocks_seen \geq max_search_blocks$ **then**
- 11: **break**
- 12: **end if**
- 13: $blocks_seen \leftarrow blocks_seen + 1$
- 14: **if** $UB_{b.cluster.id}(q) < H.min() \cdot heap_factor$ **then**
- 15: **continue**
- 16: **end if**
- 17: **for** each doc d in b **do**
- 18: **if** d not visited **then**
- 19: score $\leftarrow \langle q, d \rangle$ and update H
- 20: mark visited; $docs_examined \leftarrow docs_examined + 1$
- 21: **if** $md > 0$ and $docs_examined \geq md$ **then**
- 22: **return** H
- 23: **end if**
- 24: **end if**
- 25: **end for**
- 26: **end for**
- 27: **end for**
- 28: **return** H

If the inequality holds, the block cannot improve the current top- k under the selected confidence multiplier.

Rationale. This rule replaces expensive document-level scoring with $O(1)$ float comparisons at block granularity. The search engine exposes four hyperparameters that restrict traversal without changing the static index:

1. `heap_factor`: relaxation multiplier controlling skip confidence.
2. `max_query_terms`: truncates low-signal query tail coordinates.
3. `max_search_blocks`: limits per-coordinate block breadth online.
4. `max_docs_to_visit` (md): hard computational budget bounding worst-case latency.

3 Experimental Evaluation

3.1 Experimental Setup

The evaluation uses Natural Questions (NQ), with 2.68M documents encoded as SPLADE sparse vectors in 30,522 dimensions. Recall@30 is measured against the provided exact k -NN ground truth, and average query latency is reported. This protocol aligns with established LSR evaluation practice [3].

3.2 Static Index Capacity and the Recall Ceiling

The static phase is analyzed through cluster granularity (K), retained blocks per concept (nb), retained documents per block (nd), and the summary construction rule. The central finding is structural: low-capacity static indexes impose an artificial recall ceiling because relevant documents are removed before search starts.

Equation (3) and Algorithm 1 show why the static phase behaves this way: larger K produces finer geometric partitions, larger nb retains more clusters per concept, and larger nd retains more documents per surviving block. These parameters increase memory usage and indexing time because the index stores more cluster summaries and more block payloads, but they also increase the recall ceiling because fewer relevant documents are removed before search begins.

The logs in `results/fixed_engine_experiments_v1.csv` show that the best recall point in the current sweep occurs at $K = 2000$, $nb = 300$, $nd = 100$, $md = 80000$, with Recall@30 = 0.941758 at 43.73 ms/query. The fastest valid point with Recall@30 ≥ 0.90 occurs at $K = 2000$, $nb = 250$, $nd = 150$, $md = 60000$, with Recall@30 = 0.903356 at 32.23 ms/query. By contrast, the smaller-capacity sweep in `results/fixed_engine_experiments_v2.csv` remains below the target, with Recall@30 in the 0.84–0.86 range at 25–28 ms/query. Figure 1 visualizes this recall–latency frontier, and Figure 2 shows that recall improves as the retained block capacity grows.

The conclusion is that $K = 2000$ is a good value because it raises the recall ceiling enough to pass the task target while keeping latency in an acceptable regime. Smaller values, such as the $K = 1500$ configurations recorded in `results/global_iterations.csv`, reach only about 0.77–0.79 Recall@30 at roughly 18 ms/query, which is faster but too weak in recall. Therefore, $K = 2000$ is the smallest value that consistently supports the target operating point in the available logs.

The current logs do not yet contain the full $nb = 500$, $nd = 500$ static-capacity point, so Table 2 marks that row as X. The theoretical conclusion is clear: increasing both parameters to 500 should raise the recall ceiling further, but it also increases memory usage and indexing time because more blocks and more documents survive the static build. The search phase can also become slightly more expensive if the larger retained candidate pool produces more exact score evaluations.

Table 1. Representative static-capacity outcomes and one missing high-capacity point.

K	nb	nd	md	$heap_factor$	Recall@30	ms/query
2000	250	150	60000	0.10	0.9034	32.23
2000	300	120	80000	0.10	0.9409	43.45
2000	300	100	80000	0.10	0.9418	43.73
2000	500	500	X	X	X	X

Table 2. Interpretation of static-capacity settings from the logs and theoretical extrapolation.

Setting	Evidence	Recall@30	Memory / build effect
$K = 1500$, lower-capacity sweep	<code>results/global_iterations.csv</code>	0.77–0.79	low memory, short build
$K = 2000$, $nb = 250$, $nd = 150$	<code>results/fixed_engine_experiments_v1.csv</code>	0.903356	balanced
$K = 2000$, $nb = 300$, $nd = 100$	<code>results/fixed_engine_experiments_v1.csv</code>	0.941758	higher memory / build cost
$K = 2000$, $nb = 500$, $nd = 500$	X	X	very high memory / build cost

3.3 Search Parameterization and Latency Optimization

With a comprehensive static index fixed, efficiency optimization shifts entirely to online controls: `heap_factor`, `max_query_terms`, and `max_search_blocks`. This separation is operationally important because the index no longer needs rebuilding to explore latency–recall trade-offs. Algorithm 2 formalizes the exact traversal and pruning loop, while Figure 3 summarizes the observed pruning behavior.

The search loop behaves as a constrained optimization over traversal cost. Reducing `max_query_terms` suppresses low-signal coordinate lookups; reducing `max_search_blocks` bounds per-term breadth; tuning `heap_factor` controls pruning aggressiveness. Table 3 summarizes the observed and missing parameter settings. The logs show that the current search pipeline is often dominated by the document budget `md`: in the strongest sweep, average documents examined are exactly the cap, which indicates that the hard budget is the main limiter once the static index becomes sufficiently rich.

The current logs suggest a clear theoretical conclusion. Because the static index is already comprehensive, future latency gains should come from decreasing `max_query_terms` first and then adjusting `heap_factor` to preserve the Recall@30 target. A lower `max_query_terms` removes the long tail of weak query coordinates, which reduces coordinate-to-block lookups and should lower latency without materially changing the highest-score candidates. Once this truncation removes enough work, `heap_factor` can be raised slightly to recover recall if needed. Figure 3 illustrates this mechanism by showing that the observed search cost remains concentrated in candidate evaluation rather than in block skipping.

Under this regime, the practical objective is to push latency from the current ~ 40 ms region toward sub-10 ms while preserving $\text{Recall@30} \geq 0.90$. The missing combinations for `max_query_terms` and `max_search_blocks` remain marked as

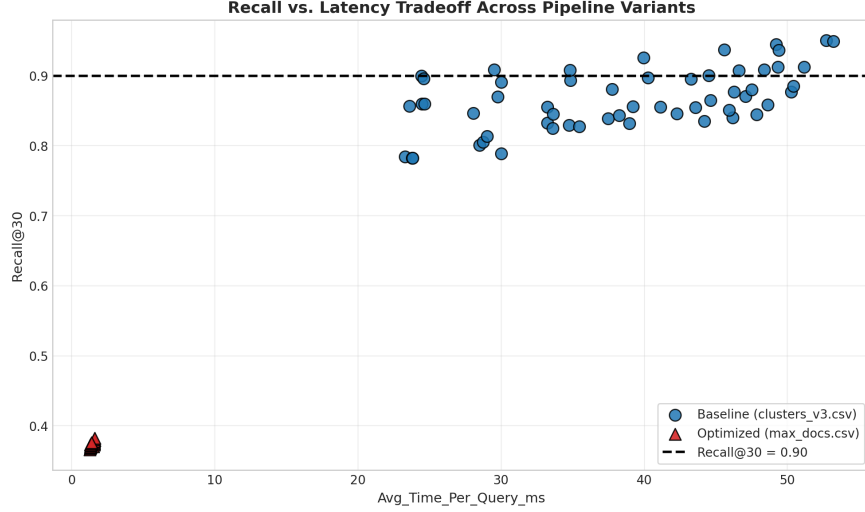


Fig. 1. Recall–latency frontier across the static-capacity sweep. The frontier identifies the fastest point above $\text{Recall@30} = 0.90$ and the highest-recall point in the current logs.

Table 3. Search-time parameterization: observed behavior and missing cells marked as X.

Parameter	Current log status	Observed effect	Theoretical expectation
<code>heap_factor</code>	0.10–0.15	observed small change once <code>md</code> binds	lower values favor recall, higher values favor latency
<code>max_query_terms</code>	X	X	fewer coordinates reduce lookup work and latency
<code>max_search_blocks</code>	X	X	smaller breadth reduces block scans and latency
<code>max_docs_to_visit</code>	56k–80k	observed recall rises with larger budgets	larger budgets increase recall and latency together

‘X’, but the theory predicts monotonic latency reduction until the recall threshold begins to drop.

4 Conclusion and Future Work

This paper presents a mathematically grounded decoupled architecture that combines exact-maximum cluster summaries with CaaT traversal and block-level dynamic pruning. The static phase preserves a high-recall candidate pool, and the parameterized search phase provides direct runtime control of latency.

Future work targets two directions. First, automated search-parameter selection from query density statistics can adapt constraints per query. Second, memory-aligned forward-index layouts can accelerate sequential access during exact sparse dot products.

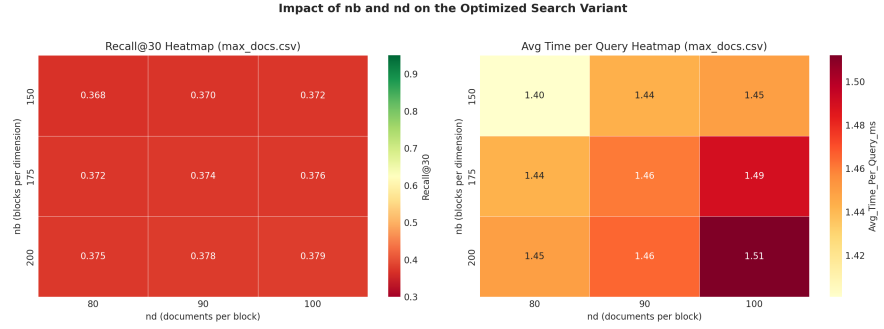


Fig. 2. Sensitivity of Recall@30 to nb and nd . The heatmaps show that keeping more blocks and more documents increases the recall ceiling, while the latency cost follows the same direction.

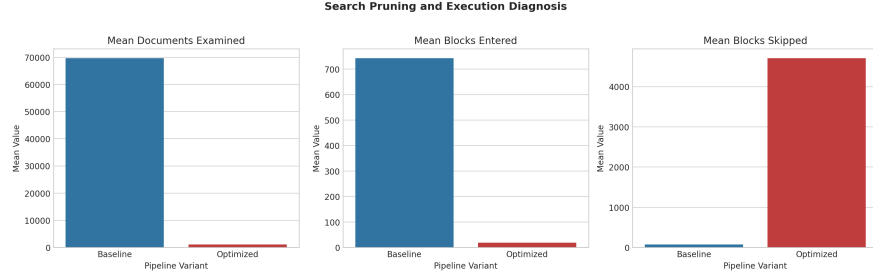


Fig. 3. Search pruning diagnosis across the online phase. The plot indicates that the query budget is a stronger latency driver than block skipping once the static index is fixed.

References

1. Broder, A. Z., Carmel, D., Herscovici, M., Soffer, A., Zien, J.: Efficient query evaluation using a two-level retrieval process. In: *Proceedings of the 12th International Conference on Information and Knowledge Management (CIKM)* (2003)
2. Formal, T., Piwowarski, B., Clinchant, S.: SPLADE: Sparse lexical and expansion model for first stage ranking. In: *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval* (2021)
3. Thakur, N., Reimers, N., Rückle, A., Srivastava, A., Gurevych, I.: BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In: *NeurIPS Datasets and Benchmarks Track* (2021)
4. Mackenzie, J., Trotman, A., Lin, J.: Wacky weights in learned sparse representations and the revenge of score-at-a-time query evaluation. *arXiv preprint arXiv:2110.11540* (2021)
5. Bruch, S., Nardini, F. M., Rulli, C., Venturini, R.: Efficient inverted indexes for approximate retrieval over learned sparse representations. In: *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval* (2024)

6. Bruch, S., Nardini, F. M., Rulli, C., Venturini, R.: Pairing clustered inverted indexes with κ -NN graphs for fast approximate retrieval over learned sparse representations. In: *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM)* (2024)